

Bancos de Dados

Aula - 06

2026

SENAI - CRTI

Operadores de Agregação no MongoDB

Unidade Curricular: Bancos de Dados

Aprenda a transformar dados brutos em informações estratégicas utilizando o **Aggregation Framework** do MongoDB — a ferramenta essencial para análise, sumarização e geração de relatórios profissionais.



Roteiro da Aula

01

Situação de Aprendizagem

Contextualização com cenário real de empresa

02

Mobilização de Conhecimentos

Ativação do que já sabemos sobre dados

03

Desenvolvimento Teórico

Aggregation Framework, estágios e operadores

04

Prática Progressiva

9 exercícios do básico ao pipeline completo

05

Sistematização e Atividade Final

Resumo visual + entregável em grupo (30 min)

O Problema da Empresa TechVendas

O Cenário

A **TechVendas** é uma empresa de e-commerce que possui milhares de registros de vendas no MongoDB. O gerente de negócios precisa responder perguntas urgentes toda semana:

- Qual produto gerou mais receita este mês?
- Qual é o ticket médio por categoria?
- Quais são os 5 produtos mais vendidos?

O Problema Real

A equipe de desenvolvimento só sabe usar `find()` e `findOne()`. Esses comandos retornam documentos individuais, mas **não conseguem agregar, somar ou calcular médias** automaticamente.

 Resultado: horas de processamento manual em planilhas, dados desatualizados e decisões lentas.



Por Que o `find()` Não Resolve?

O método `find()` é excelente para **buscar e filtrar documentos individuais**, mas não foi projetado para análise agregada. Quando precisamos de totais, médias ou agrupamentos, ele nos obriga a trazer todos os dados para a aplicação e calcular manualmente — o que é lento, ineficiente e propenso a erros.

`find()` — O que faz

Retorna documentos que correspondem a um filtro. Ideal para leitura de registros individuais.

`find()` — O que NÃO faz

Não agrupa, não soma, não calcula médias, não ordena por resultado calculado.

`aggregate()` — A Solução

Processa dados em estágios no próprio servidor, retornando resultados já calculados.

O Que Você Já Sabe Sobre Dados?

Antes de mergulharmos no Aggregation Framework, vamos ativar o conhecimento que você já possui. Pense nas situações abaixo e reflita — você provavelmente já fez isso antes, só não sabia o nome técnico!

Pergunta 1

Como você geraria um relatório de **total de vendas por mês** a partir de uma tabela com milhares de registros?

Pergunta 2

Se precisasse calcular a **média de preços** de produtos de uma categoria, como faria isso em JavaScript puro?

Pergunta 3

Já usou `GROUP BY` em SQL? O que ele faz? Como agrupar o **mesmo conceito no MongoDB**?

Pergunta 4

Como você encontraria o **produto mais caro e o mais barato** de uma coleção com apenas uma consulta?

O Aggregation Framework

O que é?

O **Aggregation Framework** é o sistema nativo do MongoDB para processar e transformar documentos de uma coleção. Ele opera através de um **pipeline** — uma sequência de estágios onde cada estágio transforma os dados e passa o resultado para o próximo.

É a principal ferramenta para **análise de dados, geração de relatórios e sumarização** no MongoDB.

Analogia do Mundo Real

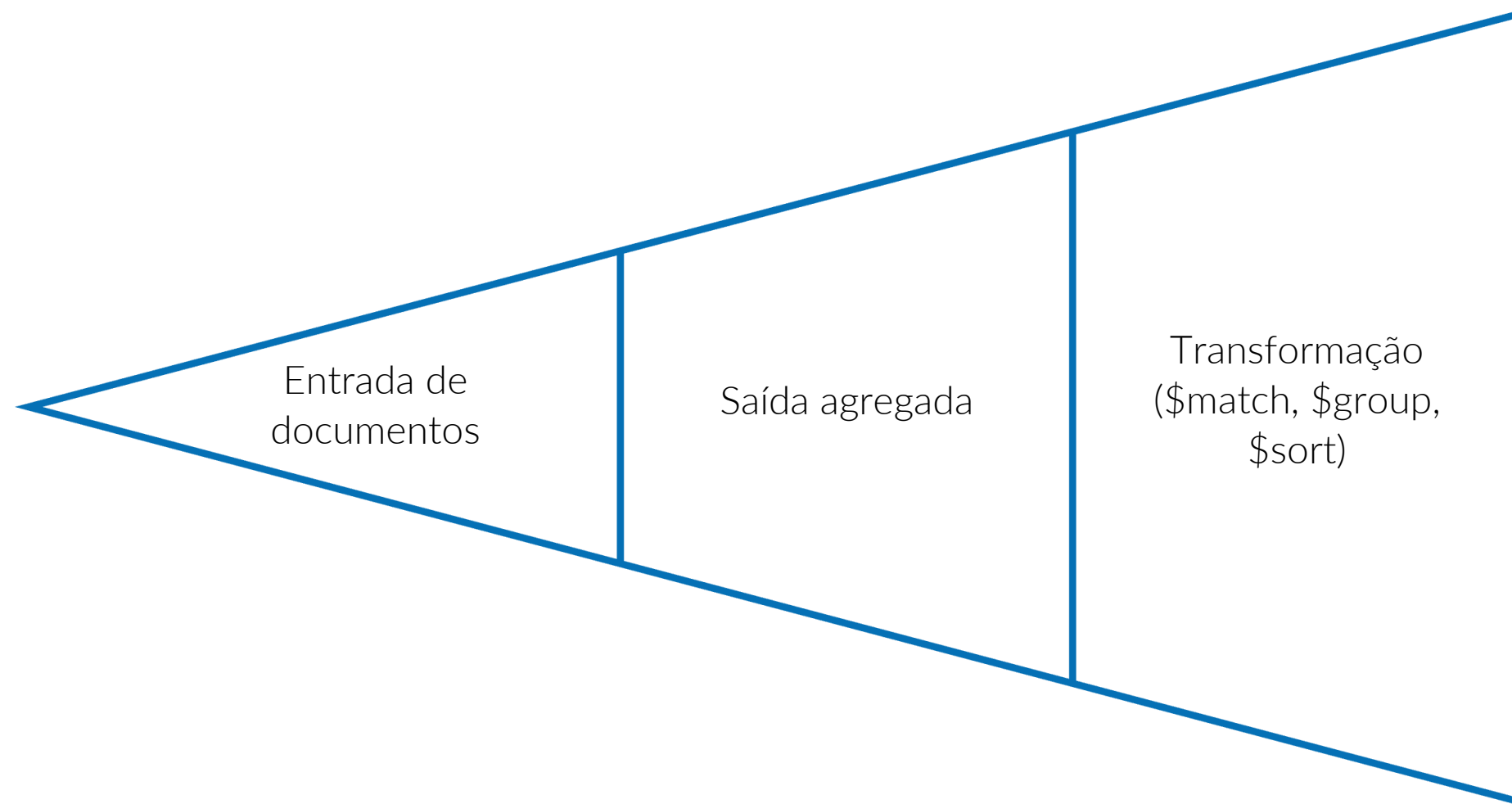
Imagine uma **linha de produção industrial**: a matéria-prima entra na esteira, passa por diferentes máquinas (estágios), e sai como produto final transformado.

-  No MongoDB: os documentos entram no pipeline, passam por estágios de transformação (\$match, \$group, \$sort...) e saem como resultado agregado.

A Estrutura do Pipeline

Um pipeline de agregação é um **array de estágios**. O MongoDB executa cada estágio em ordem, passando a saída de um como entrada para o próximo. Isso permite construir consultas complexas de forma modular e legível.

```
db.colecao.aggregate([
  { estágio1: { opções } },
  { estágio2: { opções } },
  { estágio3: { opções } }
]);
```



Cada estágio é independente e pode ser combinado livremente. Você pode ter quantos estágios precisar — do mais simples ao mais complexo.

Os 5 Estágios Principais



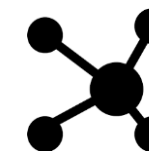
\$project — Projeção

Seleciona quais campos aparecerão no resultado. Pode incluir, excluir ou **criar novos campos calculados** a partir dos existentes.



\$match — Filtro

Filtra documentos com base em critérios, igual ao `find()`. **Deve ser usado no início** do pipeline para melhorar a performance, reduzindo o volume de dados nos próximos estágios.



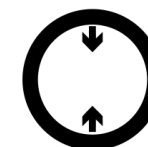
\$group — Agrupamento

Agrupa documentos por um campo (`_id`) e permite calcular acumuladores como soma, média, máximo e mínimo para cada grupo formado.



\$sort — Ordenação

Ordena os documentos por um ou mais campos. Use `1` para crescente e `-1` para decrescente. Pode ordenar por campos calculados nos estágios anteriores.



\$limit — Limite

Limita a quantidade de documentos no resultado. Muito útil para **Top 5, Top 10** ou qualquer ranking, combinado com `$sort` anterior.

Exemplo 1 — Filtro + Agrupamento

Este é o exemplo mais fundamental: **filtrar documentos de uma categoria específica** e depois somar o total de vendas por produto. Observe como o `$match` reduz os dados antes do `$group` processar.

```
db.vendas.aggregate([
  {
    $match: { categoria: "eletronicos" }
  },
  {
    $group: {
      _id: "$produto",
      total: { $sum: "$valor" }
    }
  }
]);
```

O que cada estágio faz:

- **\$match:** seleciona apenas documentos onde `categoria = "eletronicos"`
- **\$group:** agrupa por produto e soma o campo valor de cada grupo
- **_id:** define o campo de agrupamento — aqui é o nome do produto

Resultado esperado:

```
[
  { _id: "Notebook", total: 45000 },
  { _id: "Smartphone", total: 32000 },
  { _id: "Tablet", total: 18500 }
]
```

Exemplo 2 – Média Global com \$avg

Quando o `_id` recebe o valor `null`, o MongoDB trata **toda a coleção como um único grupo**, permitindo calcular métricas globais como a média geral de todos os valores.

```
db.vendas.aggregate([
  {
    $group: {
      _id: null,
      media: { $avg: "$valor" },
      total: { $sum: "$valor" },
      quantidade: { $sum: 1 }
    }
  }
]);
```

Ponto-chave: `_id: null`

Quando `_id` é `null`, não há agrupamento por campo — todos os documentos formam um único grupo global. Isso é ideal para calcular **KPIs gerais** da coleção inteira.

Resultado esperado:

```
[
  {
    _id: null,
    media: 312.87,
    total: 95425,
    quantidade: 305
  }
]
```

Note que podemos calcular **múltiplos acumuladores** no mesmo `$group`.

Exemplo 3 — Ranking com \$sort e \$limit

Um dos padrões mais usados em relatórios: **agrupar, ordenar pelo resultado calculado e limitar ao Top N**. A ordem dos estágios é crucial — o `$sort` usa o campo `total` criado pelo `$group`.

```
db.vendas.aggregate([
  {
    $group: {
      _id: "$produto",
      total: { $sum: "$valor" }
    }
  },
  {
    $sort: { total: -1 }
  },
  {
    $limit: 5
  }
]);
```

✅ Este pipeline gera um **ranking dos 5 produtos mais vendidos** em valor total — exatamente o tipo de relatório que o gerente da TechVendas precisava!

- `$sort: { total: -1 }` → ordena do maior para o menor
- `$sort: { total: 1 }` → ordena do menor para o maior
- `$limit: 5` → retorna apenas os 5 primeiros após a ordenação
- Sem o `$sort` antes, o `$limit` retorna resultados aleatórios

Exercícios Progressivos

A seguir, realizaremos **9 exercícios em ordem crescente de complexidade**. Para todos os exercícios, utilizaremos a coleção `vendas` com a estrutura abaixo. Crie esta coleção no seu ambiente antes de começar.

```
// Estrutura dos documentos da coleção "vendas"
{
  _id: ObjectId("..."),
  produto: "Notebook",
  categoria: "eletronicos",
  valor: 3500.00,
  quantidade: 2,
  vendedor: "Ana",
  data: ISODate("2024-01-15")
}
```

 Use o **MongoDB Compass** ou o **mongosh** (shell) para executar os exercícios. Insira ao menos 20 documentos de exemplo para obter resultados representativos.

Exercício 1 — Filtrar com \$match

Objetivo

Praticar o uso do `$match` para filtrar documentos antes do processamento. Entender que `$match` funciona como o `find()` dentro do pipeline.

Tarefa

Escreva um pipeline que retorne **apenas os documentos da categoria "eletronicos"**. Em seguida, modifique para filtrar também por `valor` maior que 1000.

Solução esperada:

```
db.vendas.aggregate([
  {
    $match: {
      categoria: "eletronicos",
      valor: { $gt: 1000 }
    }
  }
]);
```

Teste: quantos documentos foram retornados? Compare com `db.vendas.find({categoria:"eletronicos"}).`

Exercício 2 — Agrupar com \$group

Objetivo

Compreender o conceito de agrupamento: como transformar **muitos documentos em poucos grupos** baseados em um campo comum.

Tarefa

Agrupe todos os documentos da coleção por `categoria` e conte quantos documentos existem em cada categoria usando `$sum: 1`.

 O truque `$sum: 1` soma 1 para cada documento do grupo, efetivamente contando-os.

Solução esperada:

```
db.vendas.aggregate([
  {
    $group: {
      _id: "$categoria",
      quantidade: { $sum: 1 }
    }
  }
]);

// Resultado esperado:
[
  { _id: "eletronicos", quantidade: 8 },
  { _id: "vestuario", quantidade: 5 },
  { _id: "alimentos", quantidade: 7 }
]
```

Exercício 3 — Somar com \$sum

Objetivo

Utilizar `$sum` para acumular valores numéricos, calculando o **total de receita por produto**.

Tarefa

Calcule o **valor total vendido por produto** (some o campo `valor` agrupado por `produto`). Qual produto gerou mais receita?

Desafio extra:

Modifique o pipeline para somar `valor * quantidade`. Dica: use `$sum: { $multiply: ["$valor", "$quantidade"] }`.

Solução:

```
db.vendas.aggregate([
  {
    $group: {
      _id: "$produto",
      totalVendido: {
        $sum: "$valor"
      },
      totalComQtd: {
        $sum: {
          $multiply: [
            "$valor",
            "$quantidade"
          ]
        }
      }
    }
  }
]);
```

Exercício 4 — Calcular Média com \$avg

Objetivo

Aplicar `$avg` para calcular **médias de valores numéricos por grupo**, gerando indicadores de preço médio por categoria.

Tarefa

Calcule o **preço médio dos produtos por categoria**. Qual categoria tem o maior ticket médio?

Desafio extra:

Adicione também o `$min` e `$max` de valor ao mesmo `$group` para ter uma visão completa da faixa de preços de cada categoria.

Solução:

```
db.vendas.aggregate([
  {
    $group: {
      _id: "$categoria",
      precoMedio: {
        $avg: "$valor"
      },
      precoMinimo: {
        $min: "$valor"
      },
      precoMaximo: {
        $max: "$valor"
      }
    }
  }
]);
```

Exercício 5 — Maior e Menor Valor com \$max e \$min

Objetivo

Utilizar `$max` e `$min` para encontrar **valores extremos** dentro de grupos, auxiliando na identificação de outliers e faixas de preço.

Tarefa

Para cada **vendedor**, descubra:

- O maior valor de venda que ele realizou
- O menor valor de venda
- A quantidade total de vendas

Solução:

```
db.vendas.aggregate([
  {
    $group: {
      _id: "$vendedor",
      maiorVenda: {
        $max: "$valor"
      },
      menorVenda: {
        $min: "$valor"
      },
      totalVendas: {
        $sum: 1
      }
    }
  }
]);
// Resultado:
// { _id: "Ana", maiorVenda: 3500,
//   menorVenda: 120, totalVendas: 7 }
```

Exercício 6 — Ordenar com \$sort

Objetivo

Combinar `$group` com `$sort` para criar um **ranking baseado em valores calculados** — padrão fundamental em relatórios analíticos.

Tarefa

Calcule o total de vendas por `vendedor` e **ordene do mais vendedor para o menos vendedor**. Quem foi o campeão de vendas?

 `$sort: { campo: -1 }` → decrescente (maior primeiro)
`$sort: { campo: 1 }` → crescente (menor primeiro)

Solução:

```
db.vendas.aggregate([
  {
    $group: {
      _id: "$vendedor",
      totalVendido: {
        $sum: "$valor"
      }
    }
  },
  {
    $sort: {
      totalVendido: -1
    }
  }
]);

// Resultado (exemplo):
// { _id: "Ana", totalVendido: 18500 }
// { _id: "João", totalVendido: 14200 }
// { _id: "Maria", totalVendido: 9800 }
```

Exercício 7 — Limitar com \$limit

Objetivo

Aplicar `$limit` em conjunto com `$sort` para criar **Top N rankings** — um dos padrões mais comuns em dashboards e relatórios gerenciais.

Tarefa

Crie um pipeline que retorne o **Top 3 produtos com maior valor total de vendas**. Depois, modifique para retornar o Bottom 3 (menores valores).

 Atenção à ordem: `$sort` deve SEMPRE vir antes do `$limit` para garantir que você está limitando os corretos.

Solução:

```
// Top 3 produtos
db.vendas.aggregate([
  {
    $group: {
      _id: "$produto",
      total: { $sum: "$valor" }
    }
  },
  { $sort: { total: -1 } },
  { $limit: 3 }
]);
```

```
// Bottom 3 produtos
db.vendas.aggregate([
  {
    $group: {
      _id: "$produto",
      total: { $sum: "$valor" }
    }
  },
  { $sort: { total: 1 } },
  { $limit: 3 }
]);
```

Exercício 8 – Pipeline Completo

Agora combinamos **todos os estágios aprendidos** em um único pipeline: filtragem, agrupamento e ordenação. Este é o padrão de consulta mais usado em relatórios reais.

```
// Pipeline completo: filtro + agrupamento + ordenação
db.vendas.aggregate([
  {
    $match: {
      data: {
        $gte: ISODate("2024-01-01"),
        $lte: ISODate("2024-12-31")
      }
    },
  },
  {
    $group: {
      _id: "$categoria",
      totalReceita: { $sum: "$valor" },
      ticketMedio: { $avg: "$valor" },
      totalTransacoes: { $sum: 1 }
    }
  },
  {
    $sort: { totalReceita: -1 }
  }
]);
```

✔ **Tarefa:** Execute este pipeline e interprete os resultados. Qual categoria gerou mais receita? Qual tem o maior ticket médio? Os resultados fazem sentido com os dados inseridos?

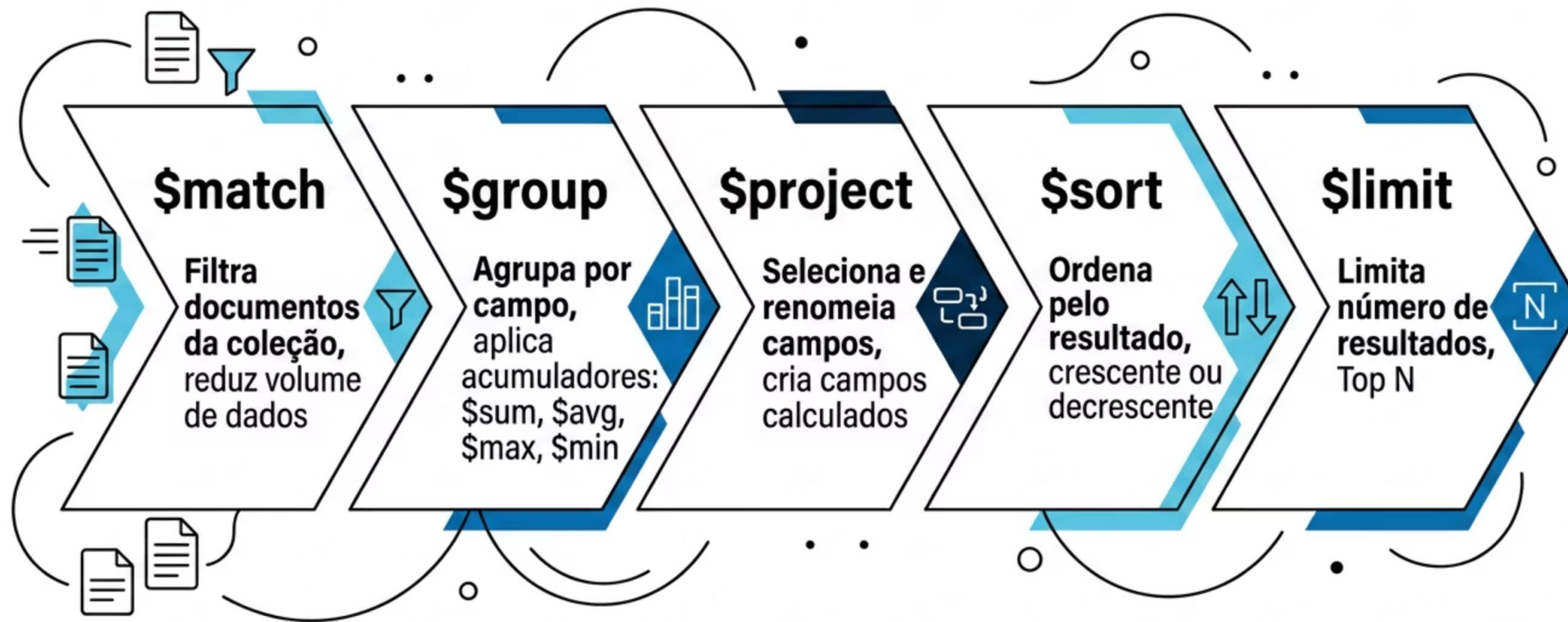
Exercício 9 — Relatório por Categoria

O exercício mais completo: gerar um **relatório analítico de vendas por categoria**, incluindo múltiplos indicadores e projeção de campos com `$project`.

```
db.vendas.aggregate([
  {
    $group: {
      _id: "$categoria",
      totalReceita: { $sum: "$valor" },
      mediaValor: { $avg: "$valor" },
      maiorVenda: { $max: "$valor" },
      menorVenda: { $min: "$valor" },
      numVendas: { $sum: 1 }
    }
  },
  {
    $project: {
      categoria: "$_id",
      totalReceita: 1,
      mediaValor: { $round: ["$mediaValor", 2] },
      maiorVenda: 1,
      menorVenda: 1,
      numVendas: 1,
      _id: 0
    }
  },
  { $sort: { totalReceita: -1 } }
]);
```

 `$project` com `$round` formata a média para 2 casas decimais. O campo `_id: 0` remove o campo `_id` padrão do resultado.

Fluxo Completo do Pipeline



A ordem dos estágios importa: sempre comece com `$match` para reduzir o volume de dados, processe com `$group`, ajuste a saída com `$project`, e por fim ordene e limite os resultados.

Resumo dos Operadores de Acumulação

Tabela de Referência Rápida

Operador	Função	Exemplo de uso
<code>\$sum</code>	Soma valores	Total de receita por produto
<code>\$avg</code>	Média aritmética	Ticket médio por categoria
<code>\$max</code>	Maior valor	Venda de maior valor do vendedor
<code>\$min</code>	Menor valor	Menor preço praticado na semana
<code>\$count</code>	Conta documentos	Número de pedidos por cliente
<code>\$sum: 1</code>	Conta documentos	Alternativa ao <code>\$count</code> no <code>\$group</code>

Regra de Ouro



Operadores de acumulação (`$sum`, `$avg`, `$max`, `$min`) devem ser usados **dentro do estágio `$group`**

Aplicações Práticas no Mundo Real



E-commerce

Total de vendas por categoria, produtos mais vendidos, receita por período, análise de sazonalidade e ranking de vendedores.



Sistema Escolar

Média de notas por turma, taxa de aprovação por disciplina, alunos com maior e menor desempenho, relatórios por professor.



Saúde

Número de atendimentos por especialidade, tempo médio de espera, distribuição de diagnósticos, relatórios mensais de procedimentos.



Finanças

Total de transações por tipo, valor médio de transferências, identificação de transações fora do padrão (máximos), fluxo de caixa por período.

PARTE 6 — ATIVIDADE FINAL

Atividade em Grupo — 30 Minutos

Chegou o momento de integrar todo o conhecimento adquirido em uma **atividade prática colaborativa**. Formem grupos de 3 a 4 alunos e sigam as instruções abaixo.



Instruções da Atividade Final

Escolha um Cenário

Selecione um dos cenários abaixo ou proponha um ao professor:

- **Vendas** — produtos, categorias, vendedores, valores
- **Escola** — alunos, disciplinas, notas, turmas
- **Biblioteca** — livros, autores, empréstimos, gêneros
- **Restaurante** — pratos, pedidos, mesas, valores

Desenvolva

1. Crie a coleção com ao menos 15 documentos de exemplo
2. Construa um pipeline com **filtro + agrupamento + ordenação**
3. Calcule: **total, média e agrupamento** (obrigatório)
4. Interprete os resultados obtidos

Entregável

Pipeline

Código do pipeline criado

Resultados

Screenshot ou cópia dos resultados

Análise

3 conclusões baseadas nos dados

Apresentação

Breve explicação para a turma (3 min)

Exemplo de Entregável – Cenário Escolar

Para inspirar os grupos, veja um exemplo completo do cenário escolar, desde a criação dos dados até a análise do resultado.

```
// 1. Criar dados de exemplo
db.notas.insertMany([
  { aluno: "Carlos", disciplina: "Matemática", nota: 8.5, turma: "T1" },
  { aluno: "Ana", disciplina: "Matemática", nota: 9.0, turma: "T1" },
  { aluno: "João", disciplina: "Português", nota: 7.5, turma: "T2" }
  // ... inserir mais 12 documentos
]);

// 2. Pipeline de análise por disciplina
db.notas.aggregate([
  { $match: { nota: { $gte: 5 } } },
  {
    $group: {
      _id: "$disciplina",
      mediaNotas: { $avg: "$nota" },
      maiorNota: { $max: "$nota" },
      menorNota: { $min: "$nota" },
      totalAlunos: { $sum: 1 }
    }
  },
  { $sort: { mediaNotas: -1 } }
]);
```



Erros Comuns e Como Evitá-los

✗ Operador fora do \$group

Usar `$sum`, `$avg`, `$max` ou `$min` fora do estágio `$group`. Esses operadores são acumuladores e **só funcionam dentro de \$group**

Boas Práticas com Aggregation Framework

1 Use `$match` o mais cedo possível

Filtrar no início do pipeline reduz drasticamente o volume de documentos processados pelos estágios seguintes, melhorando a performance.

3 Use `$project` para limpar a saída

Remova campos desnecessários e renomeie campos calculados no `$project` para tornar o resultado mais legível e próximo ao formato de relatório desejado.

2 Combine múltiplos acumuladores no mesmo `$group`

Em vez de executar vários pipelines separados, calcule `$sum`, `$avg`, `$max` e `$min` em um único estágio `$group`.

4 Teste estágio por estágio

Ao construir pipelines complexos, comece com um estágio, verifique o resultado, depois adicione o próximo. Isso facilita a identificação de erros.

Recapitulando: O Pipeline Resolveu o Problema!

O Problema Inicial da TechVendas

A equipe não sabia como gerar relatórios de total de vendas, médias de preços ou rankings de produtos a partir dos dados no MongoDB.

A Solução Encontrada

Com o Aggregation Framework, agora é possível responder qualquer pergunta de negócio em uma única consulta ao banco de dados:

```
db.vendas.aggregate([
  { $match: { ano: 2024 } },
  { $group: {
    _id: "$categoria",
    receita: { $sum: "$valor" },
    media: { $avg: "$valor" }
  }},
  { $sort: { receita: -1 } },
  { $limit: 3 }
]);
```

Capacidades Desenvolvidas ✓

- Compreensão do conceito de agregação
- Uso do Aggregation Framework
- Construção de pipelines de agregação
- Aplicação de operadores analíticos
- Interpretação de resultados



Você agora possui as ferramentas para transformar qualquer coleção MongoDB em relatórios analíticos profissionais!

